Compiling & linking C, C++ programs

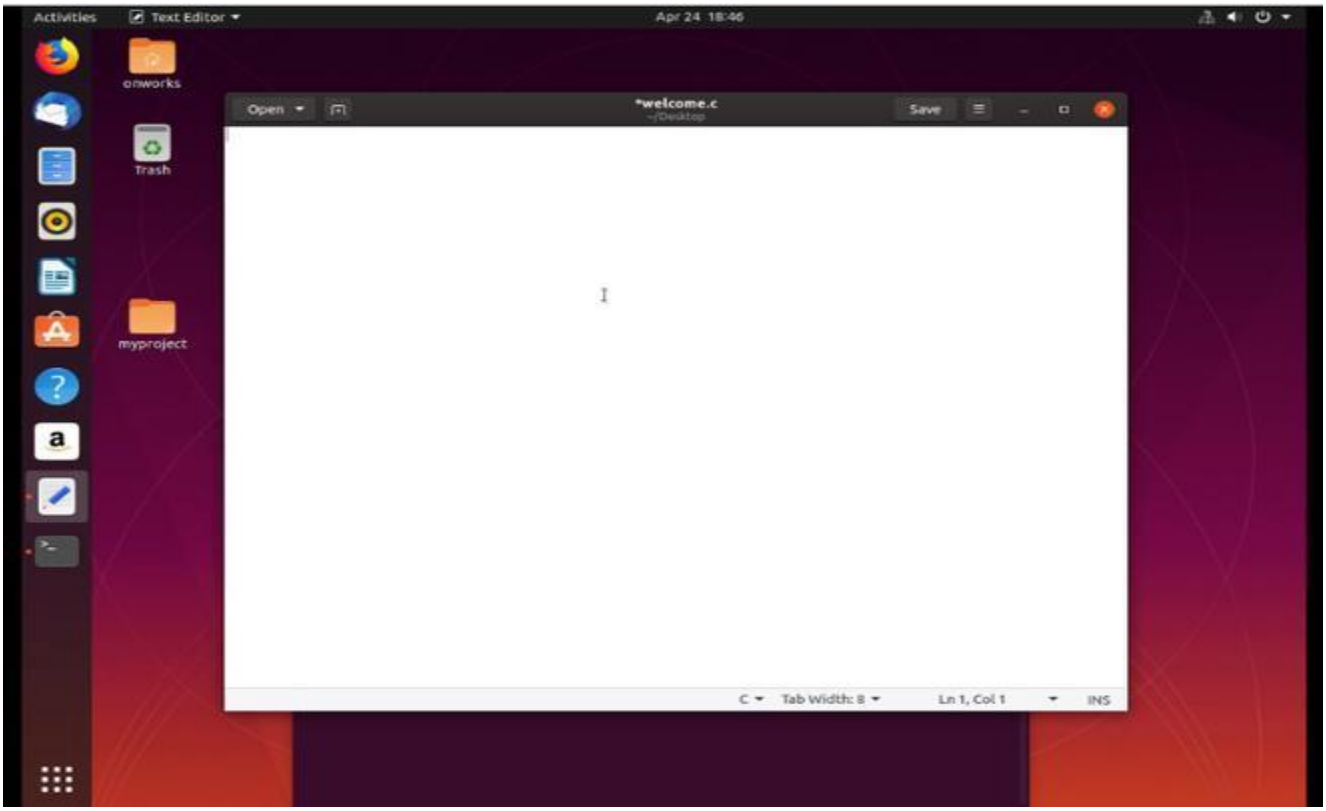
C Programming Language is mainly developed as a system programming language to write kernels or write an operating system. C++ Programming Language is used to develop games, desktop apps, operating systems, browsers, and so on because of its performance. In this article, we will be compiling and executing the C Programming Language codes and also C++ codes with various compilers like CC, GCC, and G++ compilers.

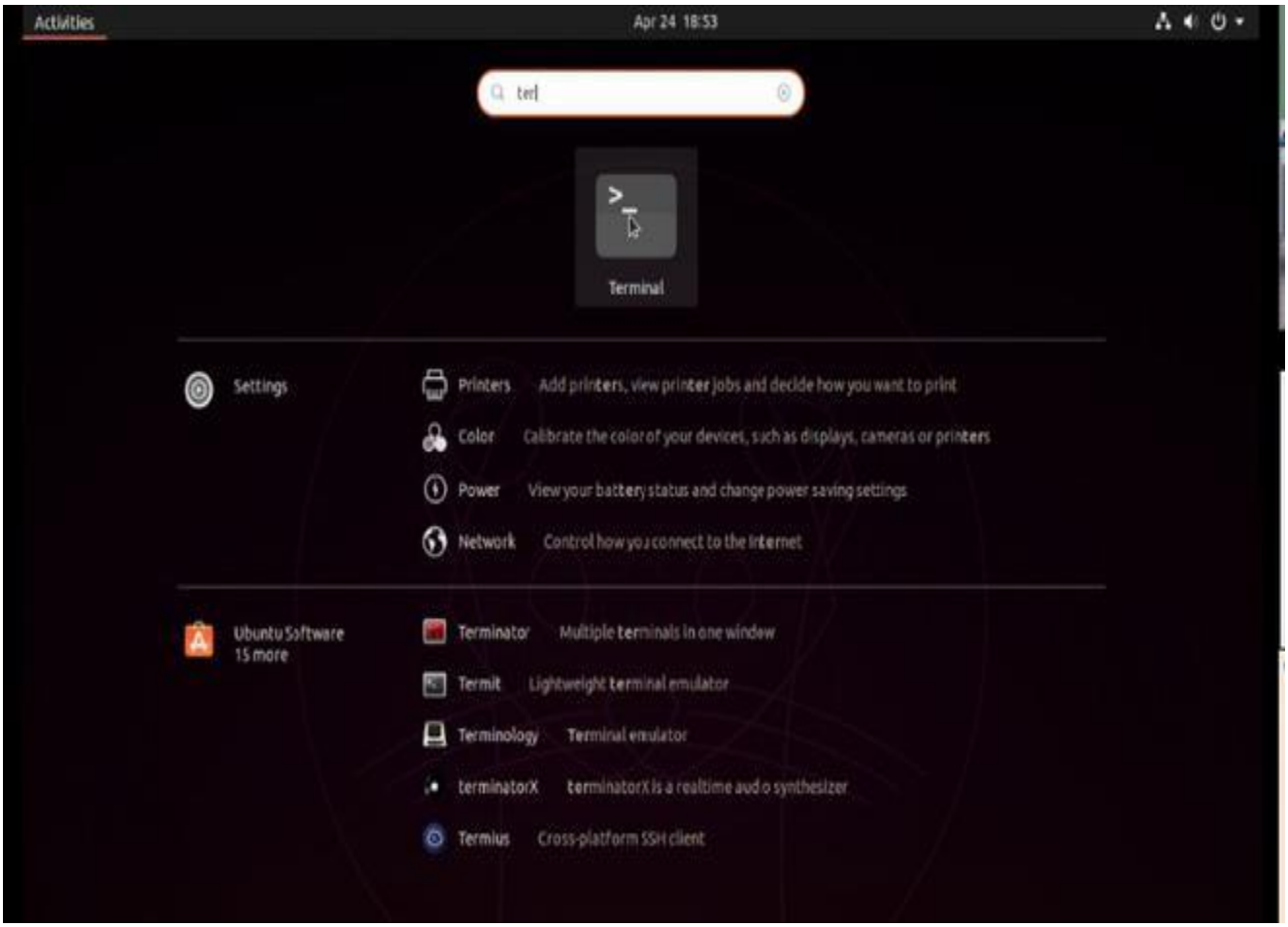
Compile & Run C Code in Linux

Method 1: Using CC Compiler

In this method, we will be compiling and executing the C program code using CC Compiler.

Step 1: First, we need to open the text editor and terminal for writing code and executing it through the terminal.





Step 2: In the text editor we need to write any code using a C programming language.

Example Script:

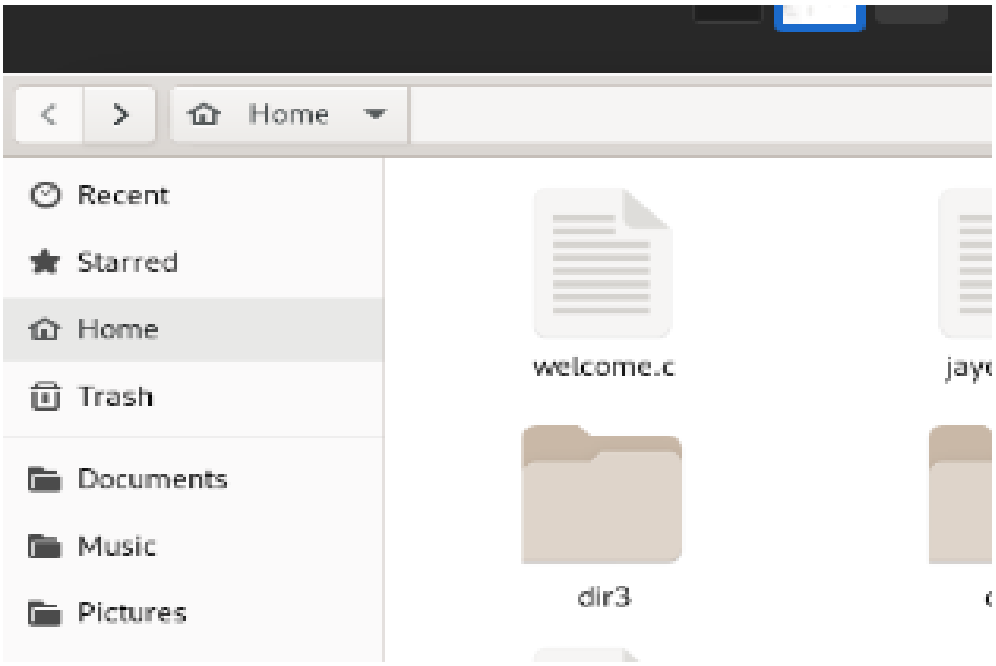
```
#include <stdio.h>

int main() {

    printf("welcome to Linux");
    return 0;
}
```

We have written a simple C program to print the "**welcome to Linux**" message.

Step 3: Now, we need to save the file with the .c extension. So in this example, we have saved the file as welcome.c.



Step 4: Now compile and run the C code in the terminal using the commands below.

```
ls
cc welcome.c
./a.out

Output
Welcome To Linux
```

cc Compiler

Our written code has been executed and the output is displayed in the above output.

Method 2: Using GCC Compiler

We can also compile and execute the script using the GCC compiler. So, let's understand the compilation and execution using GCC example below.

Example Script:

```
#include <stdio.h>

int main() {

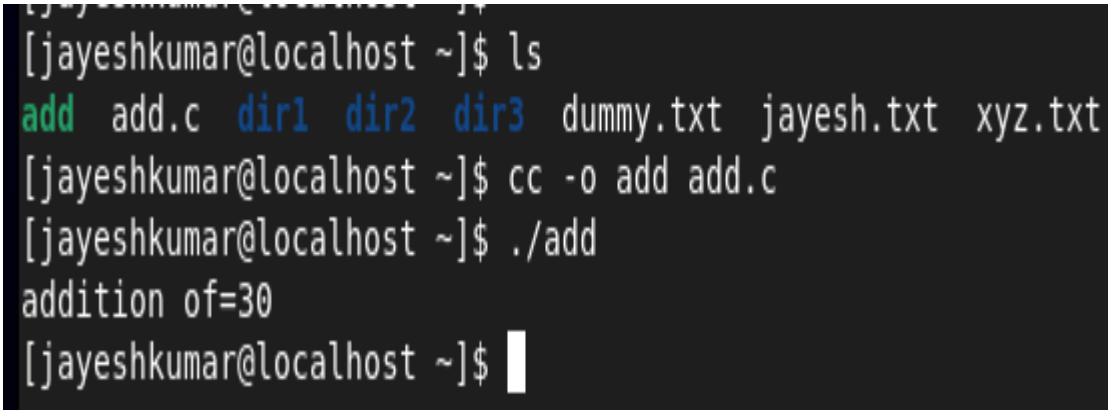
    // code
  int a=10,b=20,c;
  c=a+b;
  printf("addition of=%d\n",c);
  return 0;
}
```

Step 1: Navigate to the directory where the file is been saved. Use the below commands.

In this example we are already in the current directory.

Step 2: Execute the command below for compilation and execution.

```
cc -o add add.c
./add
```



GCC compiler

In the above image, we have written a simple C program for the addition of two numbers.

Compile & Run C++ Code in Linux

In this method, we will be compiling and executing the C++ program code using G++ Compiler.

Step 1: Write the C++ program code in a text file using a text editor and save the file with the **.cpp** extension.

Example Script:

```
#include <iostream>
using namespace std;

int main() {

    cout << "Welcome To Linux ";
    return 0;
}
```

Step 2: Navigate to the directory where the file is saved.

In this example we are already in the current directory.

Step 3: Execute the command below for compilation and execution.

```
g++ cplus.cpp
./a.out
```

```
Output
Welcome To Linux
```

g++ compiler

g++ command is a GNU c++ compiler invocation command, which is used for preprocessing, compilation, assembly and linking of source code to generate an executable file. The different "options" of g++ command allow us to stop this process at the intermediate stage.

- **Check g++ compiler version information:**

```
g++ --version
```



- **Compile a CPP file to generate executable target file:** *g++ file_name* command is used to compile and create an executable file *a.out* (default target name).

  Example: Given a simple program to print "Hello" on standard output with file name *hello.cpp*

```
// hello.cpp file
#include <iostream>
int main()
{
    std::cout << "Hello \n";
    return 0;
}
```

GDB (Step by Step Introduction)

GDB stands for GNU Project Debugger and is a powerful debugging tool for C (along with other languages like C++). It helps you to poke around inside your C programs while they are executing and also allows you to see what exactly happens when your program crashes. GDB operates on executable files which are binary files produced by the compilation process.

For demo purposes, the example below is executed on a Linux machine with the below specs.

```
uname -a
```

```
sree@ubuntu:~$ uname -a
Linux ubuntu 4.10.0-19-generic #21-Ubuntu SMP Thu Apr 6 17:04:57 UTC 2017 x86_64
 x86_64 x86_64 GNU/Linux
```

uname -a

Start GDB

Go to your Linux command prompt and type “gdb”.

gdb

Gdb open prompt lets you know that it is ready for commands. To exit out of gdb, type quit or q.

```
(gdb) quit
sree@ubuntu:~$
```

To quit

Compile the code

Below is a program that shows undefined behavior when compiled using C99.

```
sree@ubuntu:~/debugging$ cat test.c
//sample program to show undefined behaviour
//Author : Sreeraj R
#include<stdio.h>

int main()
{
    int x;
    int a = x;
    int b = x;
    int c = a + b;
    printf("%d\n", c);
    return 0;
}
```

Note: If an object that has automatic storage duration is not initialized explicitly, its value is indeterminate, where the indeterminate value is either an unspecified value or a trap representation.

Now compile the code. (here test.c). **g flag** means you can see the proper names of variables and functions in your stack frames, get line numbers and see the source as you step around in the executable. **-std=C99 flag** implies use standard C99 to compile the code. **-o flag** writes the build output to an output file.

gcc -std=c99 -g -o test test.C

Run GDB with the generated executable

Type the following command to start GDB with the compiled executable.

gdb ./test

Useful GDB commands:

Here are a few useful commands to get started with GDB.

Command	Description
run or r	Executes the program from start to end.
break or b	Sets a breakpoint on a particular line.
disable	Disables a breakpoint
enable	Enables a disabled breakpoint.
next or n	Executes the next line of code without diving into functions.
step	Goes to the next instruction, diving into the function.
list or l	Displays the code.
print or p	Displays the value of a variable.
quit or q	Exits out of GDB.
clear	Clears all breakpoints.
continue	Continues normal execution

Display the code

Now, type “l” at gdb prompt to display the code.

```
(gdb) l
1       //sample program to show undefined behaviour
2       //Author : Sreeraj R
3       #include<stdio.h>
4
5       int main()
6       {
7           int x;
8           int a = x;
9           int b = x;
10          int c = a + b;
(gdb)
```

Display the code

Set a breakpoint

Let's introduce a break point, say line 5.

```
(gdb) b 5
Breakpoint 1 at 0x5555555546a8: file test.c, line 5.
(gdb)
```

Set a breakpoint

If you want to put breakpoint at different lines, you can type "b *line_number*".By default "list or l" display only first 10 lines.

View breakpoints

In order to see the breakpoints, type "info b".

```
(gdb) info b
Num     Type           Disp Enb Address            What
1       breakpoint     keep y   0x00000000000006a8 in main at test.c:5
(gdb)
```

View breakpoints

Disable a breakpoint

Having done the above, let's say you changed your mind and you want to revert. Type "disable b".

```
(gdb) disable b
(gdb) info b
Num     Type           Disp Enb Address            What
1       breakpoint     keep n   0x00000000000006a8 in main at test.c:5
(gdb)
```

Disable a breakpoint

Re-enable a disabled breakpoint

As marked in the blue circle, Enb becomes n for disabled. 9. To re-enable the recent disabled breakpoint. Type “enable b”.

```
(gdb) enable b
(gdb) info b
Num     Type           Disp Enb Address            What
1       breakpoint     keep y   0x00000000000006a8 in main at test.c:5
(gdb)
```

Re-enable a disabled breakpoint

Run the code

Run the code by typing “run or r”.If you haven’t set any breakpoints, the run command will simply execute the full program.

```
(gdb) r
Starting program: /home/sree/debugging/test

Breakpoint 1, main () at test.c:8
8           int a = x;
(gdb)
```

Run the code

Print variable values

To see the value of variable, type “print *variable_name* or p *variable_name*“.

```
(gdb) p x
$1 = -7904
(gdb)
```

Print variable values

The above shows the values stored at x at time of execution.

Change variable values

To change the value of variable in gdb and continue execution with changed value, type “set *variable_name*“.

Installing, updating and deleting software packages

Linux offers several package managers for software installation, updates, and maintenance. In Linux, every distribution comes with a package manager by default. The Debian distribution comes with APT(Advanced Package Tool

However, there may be cases when you need to use different package managers. In this article, we'll cover the installation and setup of three prominent package managers which are:

- **DPKG: DPKG** is a base package management system used in the Debian Linux family, it is used to install, remove, store, and provide information about **'.deb'** packages. **'dpkg'** is a low-level package manager and is used in combination with higher level package managers like **'apt'** or **'apt-get',** which provide more user-friendly interfaces for package management.

- **APT: APT** is a powerful tool used in Debian-based distributions such as Ubuntu, Linux Mint, and many others. APT makes the process of installing, updating, and removing software packages simple. APT manages dependencies by itself and ensures all the packages are installed when you install a new package.

- **DNF:** DNF stands for **'Dandified YUM'** and is the default package manager of Red Hat-based Linux distributions. It is the evolution of the **YUM(Yellowdog Updater, Modified)** package manager which improves performance, dependency resolution, and usability.

Installation of a New Package Manager on Linux

Installation of different packages differs in steps at different linux distributions. Let's look at the installation of three different package managers one by one:

Installing DPKG in Debian

Package managers are core components of linux systems, and are pre-installed. To check the version of **'dpkg'** or if it is preinstalled go to Step 4 directly. However, if you need to reinstall follow the steps given below:

Step 1: Open the terminal using the keys CTRL+ALT+T or by searching the terminal in the start menu.

Step 2: To install 'dpkg' in Debian, we are using **'apt-get'** which is another package manager. In the first step, we will update the apt-get to ensure that our local package database gets synchronized with the latest updates.

```
sudo apt-get update
```

After entering the above command you need to enter the administrator password to provide root user privileges.

‘sudo’Debian**:** This command allows users with permission to execute commands with superuser(root) privileges.

Updating Packages

‘apt-get’: It is a command line package management tool in Debian-based systems. It is used for installing, removing, or updating various packages.

apt-get update

Step 3: To reinstall the **‘dpkg’** package in Debian, we need to enter the following command in the terminal:

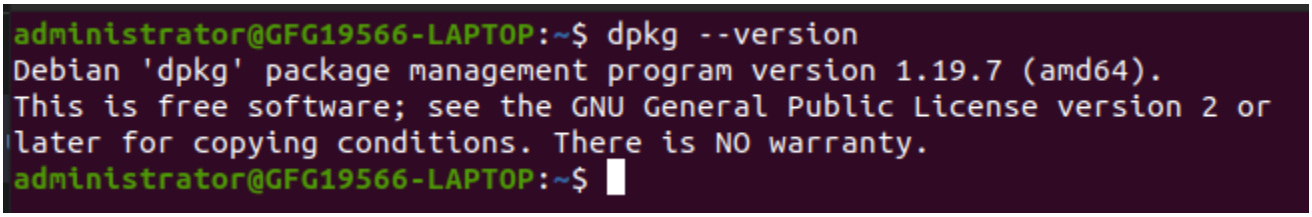
```
sudo apt --reinstall install dpkg
```

The above command tells **‘apt-get’** to reinstall the **‘dpkg’** package.

Reinstallation of dpkg

Step 3: To check if the package is installed, and the version of the package. Enter the following command:

```
dpkg --version
```



Installing APT in Ubuntu

APT is installed in Ubuntu as the default package manager. To check the version of **‘apt’** or if it is preinstalled go to Step 4 directly. However, if you need to reinstall the package use the following steps :

Step 1: Open the terminal using the keys CTRL+ALT+T or by searching the terminal in the start menu.

Step 2: To install ‘apt’ in Ubuntu. Enter the following command in the terminal to download the **‘apt’** package:

sudo apt-get update

Step 3: Install the downloaded **'apt'** package using **'dpkg':**

```
sudo apt-get -y install apt-file
```

Install the downloaded 'apt' package

Step 4: To verify if it is installed and the version of **'apt'.** Enter the following command:

apt Version

Installing DNF in RHEL(Red Hat Enterprise Linux):

DNF is installed in RHEL distributions as the default package manager. To check the version of **'dnf'** or if it is preinstalled go to Step 3 directly. However, if you need to reinstall the package use the following steps :

Step 1: Open the terminal using the keys CTRL+ALT+T or by searching the terminal in the start menu.

Step 2: We can use the **'yum'** package manager to install **'dnf':**

```
sudo yum install dnf
```

You need to enter your password to grant sudo permission.

Step 3: To verify if it is installed and the version of **'dnf'.** Enter the following command:

```
dnf --version
```

Method #1: Using the Ubuntu Software Center

One of the simplest ways to install packages in Ubuntu is to use the Ubuntu Software Center. This graphical tool allows you to browse and search for packages that are available in the official Ubuntu repositories. To use the Ubuntu Software Center, simply follow these steps:

1. Open the Ubuntu Software Center from the Applications menu.
2. Use the search field to search for the software package you want to install.
3. Select the package from the search results.
4. Click the Install button.

The Ubuntu Software Center will handle the rest, downloading and installing the package and any dependencies it requires. This method is one of the easiest ways to uninstall programs in Ubuntu.

Method #2: Using the apt Command

The apt command is a powerful package management tool that allows you to interact with the Ubuntu package repositories directly from the command line. The basic commands for using apt for installing packages are presented below:

- ***sudo apt update*** — Update the local package index with the latest information from the repositories.
- ***sudo apt install [package]*** — Install a new package.
- ***sudo apt remove [package]*** — Uninstall a package.
- ***sudo apt upgrade*** — Upgrade all packages to their latest available versions.
- ***sudo apt autoremove*** — Remove all packages that were installed automatically as dependencies and are no longer required.

Below is an example of how to install a package using the ***apt*** command:

```
sudo apt update

sudo apt install apache2
```

The above example's commands will update the local package index, and it will install the Apache web server package.

Method #3: Using a PPA (Personal Package Archive)

A PPA, or Personal Package Archive, is a collection of Ubuntu binary packages that are built and hosted by a specific individual or organization. PPAs can contain packages that are not available in the official Ubuntu repositories or they can contain packages that are in development and have not yet been added to the official repositories.

To use a PPA, you first need to add it to your system by following these steps:

1. Find the PPA you want to use.
2. Copy the PPA URL to your clipboard.
3. Open a terminal session and type the following command, replacing ***[ppa-url]*** with the URL you copied in the previous step.

```
sudo add-apt-repository [ppa-url]
```

4. Enter your password when prompted.

5. Update — using the below command — the local package index with the latest information from the new PPA:

```
sudo apt update
```

Method for Finding Installed Software on Ubuntu

One of the best and easiest ways to find installed software on Ubuntu is by using the terminal. Follow the steps below to search for installed applications using the command line.

Step #1: Open a Terminal Session

Open the terminal window by pressing ***Ctrl + Alt + T*** keys or you can open it from the applications menu.

Step #2: Run dpkg Command

The ***dpkg*** command is used to find information about installed packages. To get a list of installed applications, type the following command and press the ***Enter*** key:

```
sudo dpkg --list
```

The command above will list all the installed packages on your Ubuntu system.

Step #3: Search for Installed Software

To search for a specific installed software, use the following command:

```
sudo dpkg --list | grep <software_name>
```

For example, to find the installed version of Firefox web browser, run the following command:

```
sudo dpkg --list | grep firefox
```

More Information on Listing All Installed Software Packages on Linux

Are you curious about how to uninstall a program on Linux?

If you have an idea of what you want to uninstall, but you don't know the exact name, you can find out the exact package names in Linux. Remember, you can use the ***apt*** or ***dpkg*** command to see a list of all installed packages on your computer. Press the ***Ctrl + Alt + T*** keys to open a terminal window. Type one the following command at the prompt and press the ***Enter*** key:

```
sudo apt list --installed
sudo apt-cache search <package_name>
```

Or:

```
sudo dpkg --list
```

Scroll through the list of installed packages in the terminal window to find the one you want to uninstall. Note the full name of the package. Then, type in one of the following commands:

```
sudo apt remove <package_name>
sudo apt remove <package_name_1> <package_name_2>
sudoapt purge <package_name>
```

These commands are useful for specific situations. See the following sections for more details:

```
sudo apt-get clean
```

This command removes the aptitude cache located in ***/var/cache/apt/archives***. When you install a program, the package file is downloaded and stored in that directory. You do not need to keep the data in that directory. However, the only drawback of deleting that information is that if you decide to reinstall any of those programs again, the packages would have to be downloaded again.

Primary Methods for Uninstalling Software

Different operating systems use various commands to execute actions. To interact with the packages on Ubuntu, you will typically want to use the ***apt*** command. The ***apt*** command's primary function is to install software on Ubuntu systems and remove unwanted software. The user can choose between two options:

- ***apt-get purge***
- ***apt-get remove***

You will see this ***apt-get*** command used numerous times in the instructions for uninstalling software on Ubuntu below..

Method #1: Using the apt-get purge Command for Ubuntu Uninstalls

The ***apt-get purge*** command can be used to remove packages in Ubuntu (unnecessary packages, including configuration files, listed directories, and files). It deletes the software entirely from the system.

It's essential to back up any necessary files before running this command (example shown below). You can use the following command, substituting **<*package_name*>** with the desired package for purging:

```
sudo apt-get purge <package_name>
```

Method #2: Using the apt-get remove Command for Ubuntu Uninstalls

The ***apt-get remove*** command removes Ubuntu packages, but it does not remove configuration files and other dependencies. It only removes the package and its core components. See below for an example of the command. Use the following command, again substituting **<*package_name*>** with the desired package for removal:

```
sudo apt-get remove <package_name>
```

Method #3: Using the apt-get –f install or apt-get --fix-broken install Command to Fix Software Packages

This option uses the ***-f***or ***--fix-broken*** flags to examine the installed applications and will attempt to fix any packages it finds that are broken. These options repair broken packages and resolve any package dependencies that have inadvertently been removed. Running these options will scan the entire system for broken or missing packages. See the commands below:

```
sudo apt-get –f install
```

These flags only replace found packages that are broken and do not remove any packages. They will attempt to correct a system with broken dependencies in place. This option, when used with ***install/remove***, can omit any packages to permit ***apt*** to deduce a likely solution. If packages are specified, these have to completely correct the problem:

```
sudo apt-get --fix-broken install
```

Practice of programs

Linux Shell Script Programs / Examples

Shell Scripting is an open-source computer program that can be executed/ run on the Unix/Linux shell. **Shell Scripting** is a set of instructions to write a set of commands for the shell to execute.

A **shell script** is a set of instructions / commands (program) designed to be run by the Unix/Linux shell. It is a command-line interpreter and typical operations performed by **shell scripts** include printing text & values, file manipulation, program execution, and printing text, etc.

Program name: "swap.sh"
shell script program to swap two numbers.

num1=10
num2=20

echo "Before Swapping"
echo "Num1: $num1"
echo "Num2: $num2"

num3=$num1
num1=$num2
num2=$num3

echo "After Swapping"
echo "Num1: $num1"
echo "Num2: $num2"
Now we will save the shell script program with the "swap.sh" name.
Output:
$ sh swap.sh

Before Swapping
Num1: 10
Num2: 20
After Swapping
Num1: num2
Num2: num3

Program name: "multiply.sh"
Linux shell script program to multiply two numbers.

```
echo "Enter num1: "
read num1
echo "Enter num2: "
read num2

multiply=`expr $num1 \* $num2`
echo "Multiplication is: $multiply"
```

Now, we will save the shell script program with the "multiply.sh" name.
Output:
$ sh multiply.sh
Enter num1:
10
Enter num2:
5
Multiplication is: 50

The source code to create a Linux shell script program to demonstrate the "$#" variable is given below. The given program is compiled and executed successfully on Ubuntu 20.04.

Program name: "total_command.sh"
Linux shell script program to demonstrate the "$#" variable.

```
echo "Total command line arguments are: $#"
```

Now, we will save the shell script program with the "total_command.sh" name.
Output:
$ sh total_command.sh 1 2 3 4 5
Total command line arguments are: 5

Explanation:
In the above program, we used the $# variable to get the total number of the command-line argument passed. Here, we passed the argument total using the echo command on the console screen.

Compare two numbers

```
echo "Enter the value of A : "
read a
echo "Enter the value of B : "
read b
if [ "$a" == "$b" ]
then
echo "A is equal B"
else
echo "A is not equal to B"
```

fi

Shell Scripting for loop

1) Syntax:

```
sssit@JavaTpoint: ~
for varname in list
do
  echo "statement"
done
```

Look at the above syntax:

- Keywords are for, in, do, done
- List is a list of variables which are separated by spaces. If list is not mentioned in the for statement, then it takes the positional parameter value that were passed into the shell.
- Varname is any variable assumed by the user.

Example

```
sssit@JavaTpoint: ~
#!/bin/bash

for table in {2..20..2}
  do
    echo "table for 2 : $table"
done
```

Output

```
sssit@JavaTpoint: ~
sssit@JavaTpoint:~$ ./for.sh
table for 2 : 2
table for 2 : 4
table for 2 : 6
table for 2 : 8
table for 2 : 10
table for 2 : 12
table for 2 : 14
table for 2 : 16
table for 2 : 18
table for 2 : 20
sssit@JavaTpoint:~$
```

Look at the above snapshot, it displays the 2's table as the output.

2) Syntax of for like C programming language.

```
sssit@JavaTpoint: ~
#!/bin/bash

for (( cond1; cond2; cond3 ))

do
  echo "statement"

done
```

Example

```
sssit@JavaTpoint: ~
#!/bin/bash

for (( i=10; i >= 1; i-- ))
do
   echo "$i"
done
```

ADVERTISEMENT

Output

```
sssit@JavaTpoint: ~
sssit@JavaTpoint:~$ ./rndom.sh
10
9
8
7
6
5
4
3
2
1
sssit@JavaTpoint:~$
```

Shell Scripting while loop

Linux scripting while loop is similar to C language while loop. There is a condition in while. And commands are executed till the condition is valid. Once condition becomes false, loop terminates.

Syntax:

```
sssit@JavaTpoint: ~
while [condition]
do
   commands
done
```

Example

```
sssit@JavaTpoint: ~
i=10;
while [ $i -ge 0 ] ;
do
   echo "Reverse order number $i"
  let i--;
done
```

Output is displayed in the below

```
sssit@JavaTpoint: ~
sssit@JavaTpoint:~$ ./count.sh
Reverse order number 10
Reverse order number 9
Reverse order number 8
Reverse order number 7
Reverse order number 6
Reverse order number 5
Reverse order number 4
Reverse order number 3
Reverse order number 2
Reverse order number 1
Reverse order number 0
sssit@JavaTpoint:~$
```

while infinite loop:

Infinite loop is also called endless loop. It is made with **while true** (it means condition will always be true) or **while** : (it means an empty expression), where colon (:) is equivalent to no operation.





Shell Scripting until loop

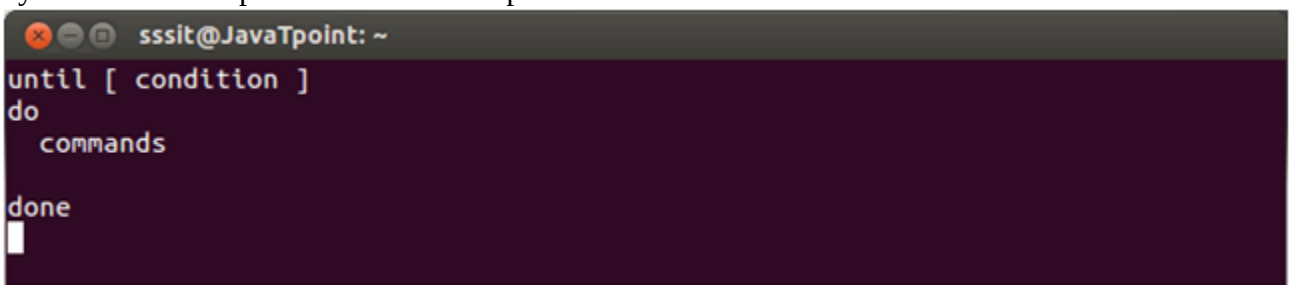
It is similar to while loop. The only difference is that **until** statement executes its code block **while** its conditional expression is false, and while statement executes its code block while its conditional expression is true.

Difference between while and until

Until loop always executes at least once. Loop while executes till it returns a zero value and until loop executes till it returns non-zero value.

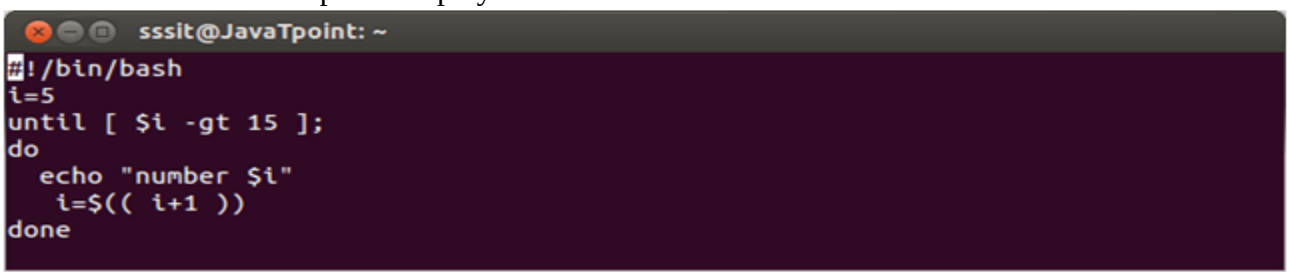
Syntax:

Syntax of until loop is shown in the snapshot below:



Example:

We have shown an example to display number from 5 to 15.



apt-get command in Linux

The command-line tool `apt-get` is the most popular package management tool used in our Debian-based Linux operating system. This article provides an overview of `apt-get` and its basic syntax. It will include the most commonly used commands, their syntax, description, and examples. It also gives an explanation of the contrast to other Linux distributions such as CentOS and Fedora, which use different package managers, and Debian-based Linux distributions use APT (Advanced Packaging Tool).

What is apt-get?

apt-get is a command-line tool that helps in handling packages in Linux. Its main task is to retrieve the information and packages from the authenticated sources for installation, upgrade, and removal of packages along with their dependencies. Here APT stands for *Advanced Packaging Tool*. It is a Debian-based Linux operating system, for example, Ubuntu and Debian itself. It is an APT (Advance Packaging Tool) system. Which manages packages for **Debian-based Linux distributions.**

What is Debian-based Linux distributions?

A family of operating systems that are built on the Debian operating system. They use the same APT (Advance Packaging Tool) for package management system. Some of the most popular Debian-based Linux distributions have different features but come under the same underlying Debian system and package management tools are Linux Mint, Ubuntu and kali Linux.

Note: apt-get (package manager) is used in Debian-based Linux distributions but in Linux distributions like CentOS and Fedora uses different package manager.

APT-GET Command in Linux

Basic Syntax:

```
sudo apt-get [options] [command] [package(s)]
```

Breakdown of each component:

- **`sudo`:** We will be using 'sudo' before every 'apt-get' command because 'sudo' in Linux system is used to execute command with the administrative privileges (root privileges). In the case of 'apt-get' which is used to manage software packages on Debian-based Linux systems, we need administrative privileges to install, remove, update and make other changes.
- **`apt-get`:** The command itself.
- **`[options]`:** This is an optional parameter in this we can use `-y` or `-s` for modifying the behavior of `apt-get` command. `-y` is for automatically answering "yes" and `-s` is for simulating a command without making any changes.
- **`[packages]`:** In this we specify the package that we want to remove, upgrade or install. If we want to mention multiple packages at same time, specify them by separated spaces between them.

Most Used Commands[APT-GET]

You need to provide one of the commands below, if the -h option is not used.

Commands	Description	Syntax	Example
update	Used in Linux-based operating systems to update the package lists for available software packages from the configured repositories.	sudo apt-get update	sudo apt-get update
upgrade	This command is used to install the latest versions of the packages currently installed on the user's system from the sources enumerated in /etc/apt/sources.list. The installed packages which have new packages available are retrieved and installed. You need to perform an update before the upgrade so that apt-get knows that new versions of packages are available.	sudo apt-get upgrade	sudo apt-get upgrade
dselect-upgrade	The command used in Linux-based operating systems to upgrade the system by installing and removing packages according to the selections made in the `dselect`.	sudo apt-get dselect-upgrade	sudo apt-get dselect-upgrade
dist-upgrade	This command performs the function of upgrade, and also handles changing dependencies with new versions of packages. If necessary, the apt-get command will try to upgrade important packages at the expense of less important ones. It may also remove some packages in this process.	sudo apt-get dist-upgrade	sudo apt-get dist-upgrade
install	This command is used to install or upgrade packages. It is followed by one or more package names the user wishes to install. All the dependencies of the desired packages will also be retrieved and installed. The user can also select the desired version by following the package name with an 'equal' and the desired version number. Also, the user can select a specific distribution by following the package name with a forward slash and the version or the archive name (e.g., 'stable', 'testing', or 'unstable'). Both of these version selection methods have the potential to downgrade the packages, so they must be used with care.	sudo apt-get install [package_name]	sudo apt-get install vim
reinstall	If we already have a package but we want to reset it to its default state, we can re-install that package.	sudo apt-get install –reinstall [package_name]	sudo apt-get install –reinstall firefox
remove	This is similar to install, with the difference being that it removes the packages instead of installing. It does not remove any configuration files created by the package.	sudo apt-get remove [package_name]	sudo apt-get remove vim
purge	This command removes the packages, and also removes any configuration files related to the packages.	sudo apt-get purge [package_name]	sudo apt-get purge vim
check	This command is used to update the package cache and check for broken dependencies.	sudo apt-get check	sudo apt-get check
download	This command is used to download the given binary package in the current directory.	sudo apt-get download [package_name]	sudo apt-get download firefox

Commands	Description	Syntax	Example
clean	This command is used to keep our system clean and tidy. It removes all the cached package files that were downloaded due to the downloading of recent packages using `apt-get`.	sudo apt-get clean	sudo apt-get clean
autoremove	Sometimes the packages which are automatically installed to satisfy the dependencies of other packages, are no longer needed then the autoremove command is used to remove this kind of packages.	sudo apt-get autoremove	sudo apt-get autoremove
source	If we want to recompile or modify the source code of a package. Source code will be downloaded in current directory in tarball format.	sudo apt-get source [package_name]	sudo apt-get source firefox
show	If we want to know more about a package, verify its installation status, version, description, other metadata and dependencies.	sudo apt-get show [package_name]	sudo apt-get show firefox
list	It also gives details (version, architecture and repository source) about package but only if package is available or installed in our system.	sudo apt-get list "or" sudo apt-get list [package_name]	sudo apt-get list firefox

Options Available in `apt-get`:

Options	Description	Syntax	Example
–no-install-recommends	It will avoid downloading any kind of recommended packages that are not strictly required by the package (installed/upgraded/reinstalled/installed). One must be caution while using this because there may be some package needed for your requirement.	sudo apt-get –no-install-recommends [Command] [package_name]	sudo apt-get –no-install-recommends install firefox
–install-suggests	It works opposite of `–no-install-recommends`. It downloads all the suggested packages that are not required much.	sudo apt-get –install-suggests [Command] [package_name]	sudo apt-get –install-suggests install firefox
-d or –download-only	In this package is only downloaded in the location (`/var/cache/apt/archives/`) by default, it is not installed, we can further in future install it.	sudo apt-get -d [command] [package_name]	sudo apt-get -d install vlc
-f or –fix-broken	By passing this option, the user specifies that apt-get should attempt to correct the system with broken dependencies in place.	sudo apt-get -f install	sudo apt-get -f install
-m or –ignore-missing or –fix-missing	By passing this option, the user specifies that apt-get should ignore the missing packages (packages that cannot be retrieved or fail the integrity check) and handle the result.	sudo apt-get -m install [package_name]	sudo apt-get -m install apache2
–no-download	In this `apt-get` is instructed to download packages already available in cache, not to download any package from configured package sources.	sudo apt-get –no-download install [package_name]	sudo apt-get –no-download install vim

Options	Description	Syntax	Example
-q or –quiet	This is used to silence the output errors or displaying warning messages. It is useful while writing scripts.	sudo apt-get -q install [package_name]	sudo apt-get -q install apache2
-s or –simulate or –just-print or –dry-run or –recon or –no-act	In this package in not actually installed, it just shows what operations it will perform while installing that package.	sudo apt-get -s install [package_name]	sudo apt-get -s install firefox
-y or –yes or –assume-yes	It automatically promotes yes while installing a package if there is any request or conformation that has to be taken from user, but one should be careful while using this.	sudo apt-get -y [command] [package_name]	sudo apt-get -y install vim
–assume-no	It automatically passes no to any question that will arise during the installation of any package.	sudo apt-get –assume-no install [package_name]	sudo apt-get –assume-no install apache2
–no-show-upgraded	Used to suppress the list of upgraded packages that is normally displayed at the end of the upgrade process. (Only works on upgrade command don't work on install, remove or purge etc).	sudo apt-get –no-show-upgraded upgrade	sudo apt-get –no-show-upgraded upgrade
-V or –verbose-versions	Used to display the version number of all the available packages including those that are not even installed. (Only work on update command, not on any other like upgrade, install, remove or purge etc)	sudo apt-get -V update	sudo apt-get -V update
–show-progress	Used to display a bar for the progress of the command. (Only used for update command and has no effect on commands like upgrade, install, remove or purge etc)	sudo apt-get –show-progress update	sudo apt-get –show-progress update
-b or –compile or –build	build-dep is used to install the build dependencies of the specified packages and `-b` is used to build a package from source.	sudo apt-get build-dep [package_name] "or" sudo apt-get -b source [package_name]	sudo apt-get build-dep firefox "or" sudo apt-get -b source vlc
–no-upgrade	It is to install a package without upgrading to a new version. (Only works with install command)	sudo apt-get install [package_name] –no-upgrade	sudo apt-get install vim –no-upgrade
–only-upgrade	It upgrades the package to the newer version while stopping the installation of other packages from being upgraded during the update process.	sudo apt-get install –only-upgrade [package_name]	sudo apt-get install –only-upgrade firefox
–reinstall	It reinstalls the package, even if the package is installed.	sudo apt-get install –reinstall [package_name]	sudo apt-get install –reinstall firefox

Options	Description	Syntax	Example
-h or –help	It is used to display a help manual.	apt-get –help	apt-get –help

Note: apt-get command will return 0 for successful executions, and decimal 100 in case of errors.

1) Difference between apt-get remove and apt-get purge?

Both of `apt-get remove` and `apt-get purge` are used for similar purposes that is to remove or uninstall the packages from our system. But still, they have a different use case.

`apt-get remove` this command only removes packages from system but leave behind their configuration files. (To see syntax, refer above context)

`apt-get purge` this command is also used to remove packages but in this case configuration files are also removed. (To see syntax, refer above context)

2)Difference between apt-get update and apt-get upgrade?

`apt-get update` This command is used to update packages rather than installing them, basically it is used to update local package index with the latest information from the configured source. Remember it does not install packages.

`apt-get upgrade` This command is used to upgrade the packages to the newest version available. Basically, it is used to upgrade and download the packages that are installed in our system.

Syntax for `apt-get update`:

```
sudo apt-get update
```

Syntax for `apt-get upgrade`:

In this we going to upgrade firefox, if it is already upgrade it will show firefox is already upgraded.

```
sudo apt-get upgrade
```

3) Not able to run `apt-get` in Red Hat Linux?

`apt-get` it is command used for package management but only in Debian-based Linux distributions like mint, Debian and ubuntu, whereas Red Hat Linux is a Linux distribution but does not come under Debian-based Linux distributions. Red Hat Enterprise Linux (RHEL), Fedora and CentOS all will not be able to use `apt-get` instead we can use `dnf` or `yum` package management tools. Just for basic example: we can use `sudo yum update` or `sudo dnf update`.

4) How to install packages using `apt-get` command?

In this we are going to install firefox package in our Debian-base Linux operating system (In this case we are executing this command in Ubuntu)

Syntax for `apt-get install`:

```
sudo apt-get install firefox
```

cvs command in Linux

In today's digital era, where file modifications and version control are essential, the Concurrent Versions System (CVS) command in Linux emerges as a powerful tool. CVS allows users to store and track the history of files, enabling easy retrieval of previous versions and restoring corrupted files. In this article, we will delve into the intricacies of the CVS command, exploring its syntax, policy options, essential commands, and practical examples.

Syntax of `cvs` command in Linux

```
cvs [cvs_options] cvs_command [command_options] [command_args]
```

CVS offers a set of policy options that enhance its functionality. These options are included.

Options	Description
–allow-root=rootdir	Specify repository on the command line. It also specifies the legal cvsroot directory. See 'Password authentication server' in the CVS manual.
-d, cvs_root_directory	It uses cvs_root_directory as the directory path name of the repository. It also overrides the *$CVSROOT* environment variable.
-e, editor-command	It uses the editor command specified for entering log information. It also overrides *$CVSEDITOR* and *$EDITOR* environment variables.
–f	It does not read the *~/.cvsrc* file.
-H	It displays CVS command help.
-n	It does not make any changes to the root repository and also prints out what would happen if the "-n" flag was not used.
-Q	Quiet mode. Less verbose than normal.
-q	Marginally quiet mode. Reports of recursion are suppressed.
-v	Show CVS software version and copyright information.
-w	Make new working files read-write. Overrides the setting of the *$CVSREAD* environment variable.

Essential CVS Commands

CVS encompasses a comprehensive set of commands to manage file history effectively. Some key commands are included.

Options	Description
add	Add a new file/directory to the repository.
admin	Administration front-end for RCS.
annotate	Shows the last revision were
checkout	Checkout sources for editing.
commit	Check files into the repository.

Options	Description
diff	Show differences between revisions
edit	Get ready to edit a watched file
editors	See who is editing a watched file.
export	Export sources from CVS, similar to checkout.
history	Show repository access history.
import	Import sources into CVS, using vendor branches.
init	It creates a CVS repository if it doesn't exist.
log	Print out historical information for files.
rdiff	Create 'patch' format diffs between revisions.
status	Display status information on checked out files.
tag	It adds a symbolic tag to check out version of files.
unedit	Undo an edit command.
update	Bring work tree in sync with repository.
version	Show current CS version(s).
watch	Set watches.

Difference between Concurrent Versions System (CVS) and Subversion (SVN)

Concurrent Versions System (CVS) :
Concurrent versions System is functional version control system which is developed by Dick Grune as series of shell scripts. This helps the teams to be connected to the changes that are measured into repository when working on software. This tool was used as the version control system for long for time. It is reliable software tool but with new challenges other alternatives made it used as limited.

Following are some features of CVS :

- It is one of the reliable Version Control System
- It does not allows commit with errors to made.
- Scripts are written in RCS.
- User can only store files into repository.

Advantages :

- CVS is one of the reliable version control software.
- Changes are committed with full change.

Disadvantages :

- CVS changes are time consuming.
- CVS do not commit if there is error into the commit.

2. Subversion (SVN) :
Subversion is an open-source highly functional version control system which is developed by CollabNet Inc and then later was taken by Apache Software Foundation. This system has a centralized control by a server. This server is responsible for the storage of repository, and through the server, this repository is then distributed into three areas called Branch, Area, and Truck. This makes the SVN to be different.

Following are some features of SVN :

- It binds with a diverse number of programming languages.
- When multiple people access the same file it is locked called File Locking.
- Directives and editing are versioned.
- Binary files are handled with care.

Advantages :

- SVN is highly configurable.
- Latest changes are committed and observed only.

Disadvantages :

- SVN central repository makes commit changes time consuming.
- It is difficult to learn SVN for the first timers.

RCS

The Revision Control System (RCS) is a software implementation of revision control that automates the storing, retrieval, logging, identification, and merging of revisions. RCS is useful for text that is revised frequently, for example programs, documentation, procedural graphics, papers, and form letters. RCS is also capable of handling binary files, though with reduced efficiency and efficacy. Revisions are stored with the aid of the diff utility.
RCS is great for small projects or managing the wealth of configuration files in `/etc`. It provides a way to reduce the need for file backups(without losing any backup quality or quantity), to

RCS really is that simple to use, it does have many more options that are not covered here(see the man pages) but the power of RCS is how simple it is to use. It takes almost no time to setup, and probably less time then you currently spend on arranging backups. The command syntax is simple, and stable across the separate commands. It provides an immensely powerful way to control configuration files, source code.

AWK command

Awk is a scripting language used for manipulating data and generating reports. The awk command programming language requires no compiling and allows the user to use variables, numeric functions, string functions, and logical operators.

Awk is a utility that enables a programmer to write tiny but effective programs in the form of statements that define text patterns that are to be searched for in each line of a document and the action that is to be taken when a match is found within a line. Awk is mostly used for pattern scanning and processing. It searches one or more files to see if they contain lines that matches with the specified patterns and then perform the associated actions.

Awk is abbreviated from the names of the developers – Aho, Weinberger, and Kernighan.

WHAT CAN WE DO WITH AWK?

1. AWK Operations:
(a) Scans a file line by line
(b) Splits each input line into fields
(c) Compares input line/fields to pattern
(d) Performs action(s) on matched lines

2. Useful For:
(a) Transform data files
(b) Produce formatted reports

3. Programming Constructs:
(a) Format output lines
(b) Arithmetic and string operations
(c) Conditionals and loops

Syntax:
awk options 'selection _criteria {action }' input-file > output-file
Options:
-f program-file : Reads the AWK program source from the file
program-file, instead of from the first command line argument.
-F fs : Use fs for the input field separator
Sample Commands

Example:

Consider the following text file as the input file for all cases below:

```
$cat > employee.txt
ajay manager account 45000
sunil clerk account 25000
varun manager sales 50000
amit manager account 47000
tarun peon sales 15000
deepak clerk sales 23000
sunil peon sales 13000
satvik director purchase 80000
```

1. Default behavior of Awk: By default Awk prints every line of data from the specified file.

$ awk '{print}' employee.txt

Output:

```
ajay manager account 45000
sunil clerk account 25000
varun manager sales 50000
amit manager account 47000
tarun peon sales 15000
deepak clerk sales 23000
sunil peon sales 13000
satvik director purchase 80000
```

In the above example, no pattern is given. So the actions are applicable to all the lines. Action print without any argument prints the whole line by default, so it prints all the lines of the file without failure.

2. Print the lines which match the given pattern.

```
$ awk '/manager/ {print}' employee.txt
```

Output:

```
ajay manager account 45000
varun manager sales 50000
amit manager account 47000
```

In the above example, the awk command prints all the line which matches with the ‘manager’.

3. Splitting a Line Into Fields : For each record i.e line, the awk command splits the record delimited by whitespace character by default and stores it in the $n variables. If the line has 4 words, it will be stored in $1, $2, $3 and $4 respectively. Also, $0 represents the whole line.

```
$ awk '{print $1,$4}' employee.txt
```

Output:

```
ajay 45000
sunil 25000
varun 50000
amit 47000
tarun 15000
deepak 23000
sunil 13000
satvik 80000
```

In the above example, $1 and $4 represents Name and Salary fields respectively.

Built-In Variables In Awk

Awk's built-in variables include the field variables—$1, $2, $3, and so on ($0 is the entire line) — that break a line of text into individual words or pieces called fields.

- **NR:** NR command keeps a current count of the number of input records. Remember that records are usually lines. Awk command performs the pattern/action statements once for each record in a file.
- **NF:** NF command keeps a count of the number of fields within the current input record.
- **FS:** FS command contains the field separator character which is used to divide fields on the input line. The default is "white space", meaning space and tab characters. FS can be reassigned to another character (typically in BEGIN) to change the field separator.
- **RS:** RS command stores the current record separator character. Since, by default, an input line is the input record, the default record separator character is a newline.
- **OFS:** OFS command stores the output field separator, which separates the fields when Awk prints them. The default is a blank space. Whenever print has several parameters separated with commas, it will print the value of OFS in between each parameter.
- **ORS:** ORS command stores the output record separator, which separates the output lines when Awk prints them. The default is a newline character. print automatically outputs the contents of ORS at the end of whatever it is given to print.

Examples:

Use of NR built-in variables (Display Line Number)

```
$ awk '{print NR,$0}' employee.txt
```

Output:

```
1 ajay manager account 45000
2 sunil clerk account 25000
3 varun manager sales 50000
4 amit manager account 47000
5 tarun peon sales 15000
6 deepak clerk sales 23000
7 sunil peon sales 13000
8 satvik director purchase 80000
```

In the above example, the awk command with NR prints all the lines along with the line number.

Use of NF built-in variables (Display Last Field)

```
$ awk '{print $1,$NF}' employee.txt
```

Output:

```
ajay 45000
sunil 25000
varun 50000
amit 47000
tarun 15000
deepak 23000
sunil 13000
satvik 80000
```

In the above example $1 represents Name and $NF represents Salary. We can get the Salary using $NF , where $NF represents last field.

Another use of NR built-in variables (Display Line From 3 to 6)

```
$ awk 'NR==3, NR==6 {print NR,$0}' employee.txt
```

Output:

```
3 varun manager sales 50000
4 amit manager account 47000
5 tarun peon sales 15000
6 deepak clerk sales 23000
```

1) To print the first item along with the row number(NR) separated with " – " from each line in geeksforgeeks.txt:

```
$ awk '{print NR "- " $1 }' geeksforgeeks.txt
1 - A
2 - Tarun
3 – Manav
4 - Praveen
```

2) To return the second column/item from geeksforgeeks.txt:

The question should be:- To return the second column/item from geeksforgeeks.txt:

```
$ awk '{print $2}' geeksforgeeks.txt
B
A12
B6
M42
```

3) To print any non empty line if present

```
$ awk 'NF < 0' geeksforgeeks.txt
```

here NF should be 0 not less than and the user have to print the line number also:

correct answer : awk 'NF == 0 {print NR}' geeksforgeeks.txt

OR

awk 'NF <= 0 {print NR}' geeksforgeeks.txt

```
0
```

4) To find the length of the longest line present in the file:

```
$ awk '{ if (length($0) > max) max = length($0) } END { print max }'
geeksforgeeks.txt
13
```

5) To count the lines in a file:

```
$ awk 'END { print NR }' geeksforgeeks.txt
3
```

6) Printing lines with more than 10 characters:

```
$ awk 'length($0) > 10' geeksforgeeks.txt
Tarun    A12    1
Praveen    M42    3
```

7) To find/check for any string in any specific column:

```
$ awk '{ if($3 == "B6") print $0;}' geeksforgeeks.txt
```

8) To print the squares of first numbers from 1 to n say 6:

```
$ awk 'BEGIN { for(i=1;i<=6;i++) print "square of", i, "is",i*i; }'
square of 1 is 1
square of 2 is 4
square of 3 is 9
square of 4 is 16
square of 5 is 25
square of 6 is 36
```

Programs for printing different patterns

Given the number N which represents the number of rows and columns, print the different following patterns.

Pattern-1:

```
Input: 6
Output:
     #
    ##
   ###
  ####
 #####
######
```

Use nested loop to print the given pattern. The first loop represents the row and the second loop represents the column.

```
# Program to print the
# given pattern

# Static input for N
N=5

# variable used for
# while loop
i=0
j=0

while [ $i -le `expr $N - 1` ]
do
    j=0

    while [ $j -le `expr $N - 1` ]
    do
        if [ `expr $N - 1` -le `expr $i + $j` ]
        then
          # Print the pattern
          echo -ne "#"
        else
          # Print the spaces required
          echo -ne " "
        fi
        j=`expr $j + 1`
    done
    # For next line
    echo

    i=`expr $i + 1`
done
```

Output:

```
    #
   ##
  ###
 ####
#####
```

Pattern-2:

```
Input: 3
Output:
    #
```

```
   ###
  #####
```

Use nested loops to print the left part and right part of the pattern. The details are explained in the code:

```
# Program in Linux to
# print pyramid

# Static input to the
# number
p=7;

for((m=1; m<=p; m++))
do
    # This loop print spaces
    # required
    for((a=m; a<=p; a++))
    do
      echo -ne " ";
    done

    # This loop print the left
    # side of the pyramid
    for((n=1; n<=m; n++))
    do
      echo -ne "#";
    done

    # This loop print right
    # side of the pyramid.
    for((i=1; i<m; i++))
    do
      echo -ne "#";
    done

    # New line
    echo;
done
```

Output:

```
      #
     ###
    #####
   #######
  #########
 ###########
#############
```

Shell Script to print odd and even numbers

```
# Take user input
echo "Enter a number"
read n

echo "Even Numbers - "
i=1

# -le means less than or equal to
while [ $i -le $n ]
do
```

```
# arithmetic operations is performed with 'expr'
rs=`expr $i % 2`

if [ $rs == 0 ]
then
echo "  $i"

# end of if loop
fi

# incrementing i by one
((++i))

# end of while loop
done
# Now printing odd numbers
echo "Odd Numbers - "
i=1
while [ $i -le $n ]
do
rs=`expr $i % 2`

if [ $rs != 0 ]
then
echo "  $i"
fi

((++i))
done
```

Output:

```
Enter a number
10
Even Numbers -
  2
  4
  6
  8
  10
Odd Numbers -
  1
  3
  5
  7
  9
```

Linux Shell - The for Loop

```
for var in 0 1 2 3 4 5 6 7 8 9
do
   echo $var
done
```

Upon execution, you will receive the following result −

```
0
1
2
3
4
5
6
7
8
9
```

Array Basics Shell Scripting (Using Loops)

```
# To declare static Array
arr=(1 12 31 4 5)
i=0

# Loop upto size of array
# starting from index, i=0
while [ $i -lt ${#arr[@]} ]
do
    # To print index, ith
    # element
    echo ${arr[$i]}

    # Increment the i = i + 1
    i=`expr $i + 1`
done
```

Output:

```
1
2
3
4
5
```

```
# To declare static Array
arr=(1 2 3 4 5)

# loops iterate through a
# set of values until the
# list (arr) is exhausted
for i in "${arr[@]}"
do
    # access each element
    # as $i
    echo $i
done
```

Output:

```
1
2
3
4
5
```

Read the array elements at run time and then Print the Array. 1. Using While-loop

```
# To input array at run
# time by using while-loop

# echo -n is used to print
# message without new line
echo -n "Enter the Total numbers :"
read n
echo "Enter numbers :"
i=0

# Read upto the size of
# given array starting from
# index, i=0
while [ $i -lt $n ]
do
    # To input from user
    read a[$i]

    # Increment the i = i + 1
    i=`expr $i + 1`
done

# To print array values
# starting from index, i=0
echo "Output :"
i=0

while [ $i -lt $n ]
do
    echo ${a[$i]}

    # To increment index
    # by 1, i=i+1
    i=`expr $i + 1`
done
Enter the Total numbers :3
Enter numbers :
1
3
5
Output :
1
3
5
```

Using for-loop

```
# To input array at run
# time by using for-loop

echo -n "Enter the Total numbers :"
read n
echo "Enter numbers:"
i=0

# Read upto the size of
# given array starting
```

```
# from index, i=0
while [ $i -lt $n ]
do
    # To input from user
    read a[$i]

    # To increment index
    # by 1, i=i+1
    i=`expr $i + 1`
done

# Print the array starting
# from index, i=0
echo "Output :"

for i in "${a[@]}"
do
    # access each element as $i
    echo $i
done
```

Output:

```
Enter the Total numbers :3
Enter numbers :
1
3
5
Output :
1
3
5
```